

Garbage in, garbage out (GIGO)

In this video, you will learn

- That having good, clean data is important.
- That having clean data requires human expertise.



xkcd.com/1838/

$$\text{PRECISE NUMBER} + \text{PRECISE NUMBER} = \text{SLIGHTLY LESS PRECISE NUMBER}$$

$$\text{PRECISE NUMBER} \times \text{PRECISE NUMBER} = \text{SLIGHTLY LESS PRECISE NUMBER}$$

$$\text{PRECISE NUMBER} + \text{GARBAGE} = \text{GARBAGE}$$

$$\text{PRECISE NUMBER} \times \text{GARBAGE} = \text{GARBAGE}$$

$$\sqrt{\text{GARBAGE}} = \text{LESS BAD GARBAGE}$$

$$(\text{GARBAGE})^2 = \text{WORSE GARBAGE}$$

$$\frac{1}{N} \sum (\text{N PIECES OF STATISTICALLY INDEPENDENT GARBAGE}) = \text{BETTER GARBAGE}$$

$$(\text{PRECISE NUMBER})^{\text{GARBAGE}} = \text{MUCH WORSE GARBAGE}$$

$$\text{GARBAGE} - \text{GARBAGE} = \text{MUCH WORSE GARBAGE}$$

$$\frac{\text{PRECISE NUMBER}}{\text{GARBAGE} - \text{GARBAGE}} = \text{MUCH WORSE GARBAGE, POSSIBLE DIVISION BY ZERO}$$

$$\text{GARBAGE} \times 0 = \text{PRECISE NUMBER}$$

xkcd.com/2295/

More on GIGO

Awful data leads to awful results.

Unfortunately, awesome data does not guarantee awesome results.

When possible, start with clean data.

Be meticulous, critical, patient, and unbiased when cleaning data.

Z-score normalization

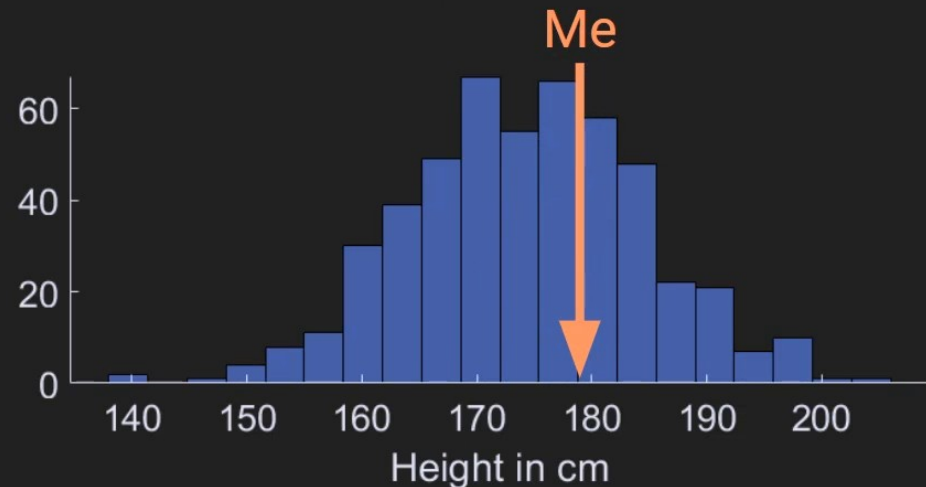
In this video, you will learn

- How to transform data to normalized z-score.
- Why the z-score transform is so awesome and why it is so ubiquitous.
- How to interpret the z-score.
- The assumptions of the z-score transform.

Why we need z: Apples vs. oranges

How does my height compare to someone else's weight?

Problem: The units are completely different and therefore totally incomparable.



Why we need z: Apples vs. oranges

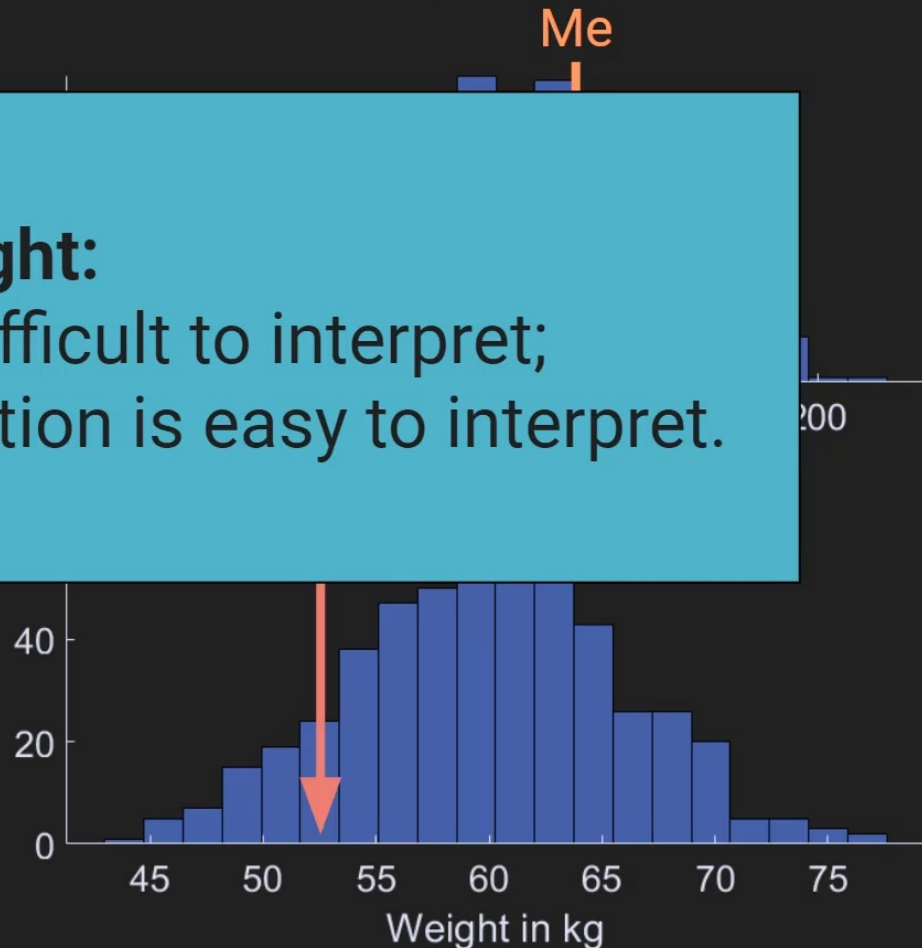
How does my weight compare to
some

Prob
com
there

Solution: Normalize both
measurements to a unit-less
scale!

Key insight:

A value on its own is difficult to interpret;
a value relative to its distribution is easy to interpret.



Z-transform

Mean-center: Subtract the average from each individual value.

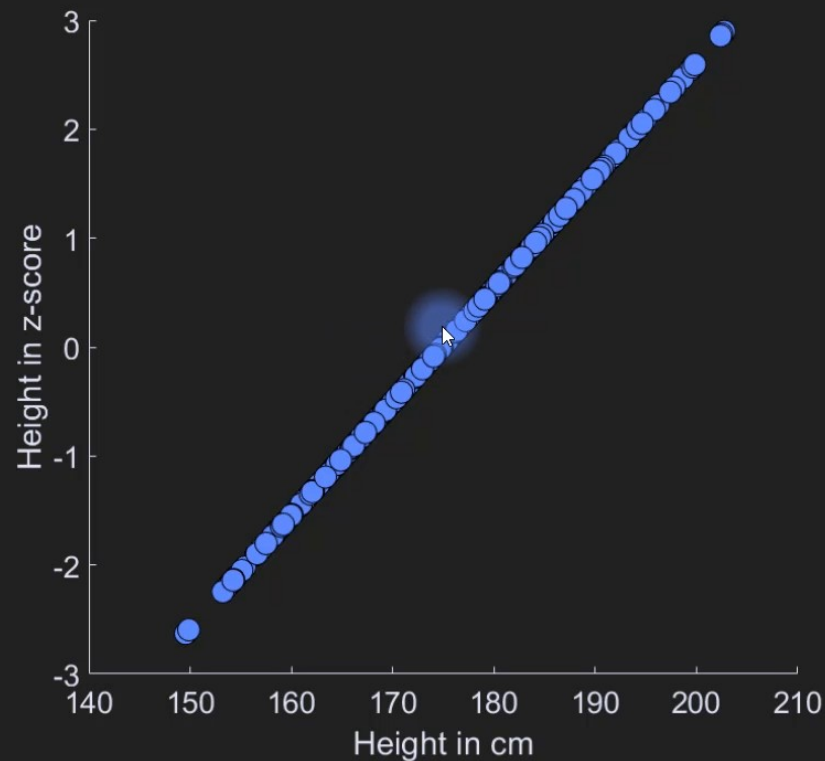
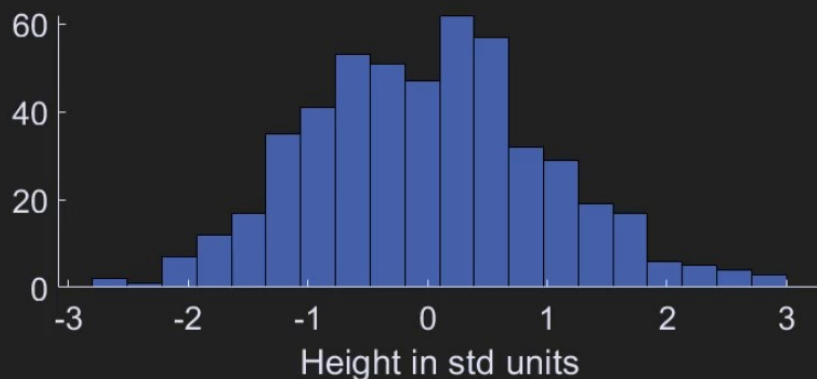
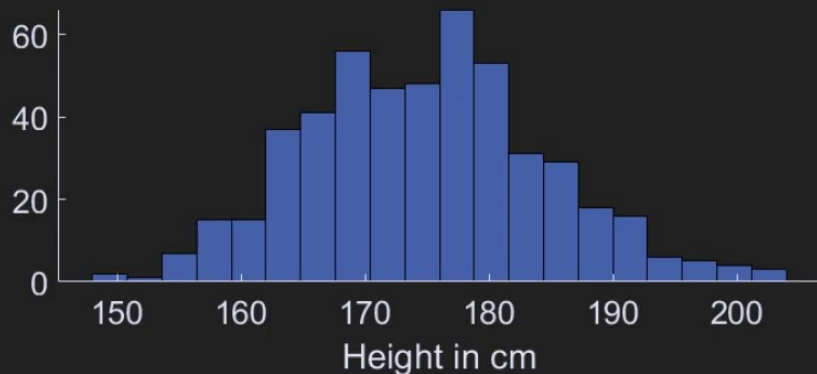
Variance-normalize: Divide by the standard deviation.

The units are standard deviations away from the mean of the distribution.

$$z_i = \frac{x_i - \bar{x}}{\sigma_x}$$

Z-transform

Z-transform shifts and stretches, but doesn't change shape.



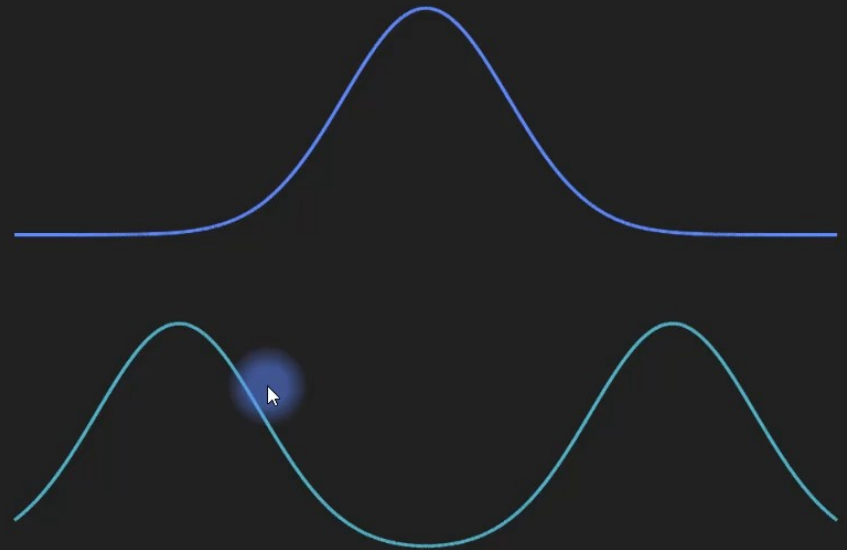
Z-transform assumption

What is the key assumption that makes the z-transform valid?

Mean and standard deviation are valid descriptions of the distribution's central tendency and dispersion.

That is, the distribution is *roughly* Gaussian.

$$z_i = \frac{x_i - \bar{x}}{\sigma_x}$$



```

import matplotlib.pyplot as plt
import numpy as np
from scipy import stats

## create data
data = np.random.poisson(3,1000)**2
## compute the mean and std
datamean = np.mean(data)
datastd = np.std(data,ddof=1)
# the previous two lines are equivalent to the following two lines
#datamean = data.mean()
#datastd = data.std(ddof=1)

plt.plot(data,'s',markersize=3)
plt.xlabel('Data index')
plt.ylabel('Data value')
plt.title(f'Mean = {np.round(datamean,2)}; std = {np.round(datastd,2)}')

plt.show()

## now for z-scoring
# z-score is data minus mean divided by stdev
dataz = (data-datamean) / datastd
# can also use Python function
dataz = stats.zscore(data)
# compute the mean and std
datazmean = np.mean(dataz)
datazstd = np.std(dataz,ddof=1)

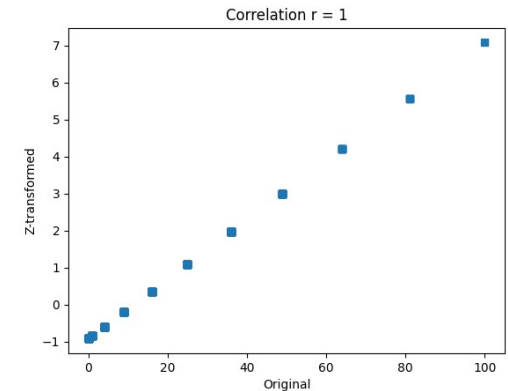
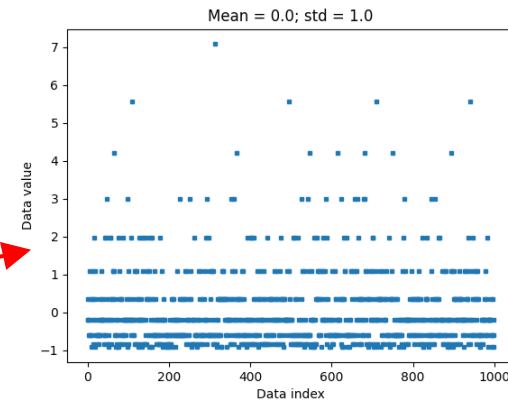
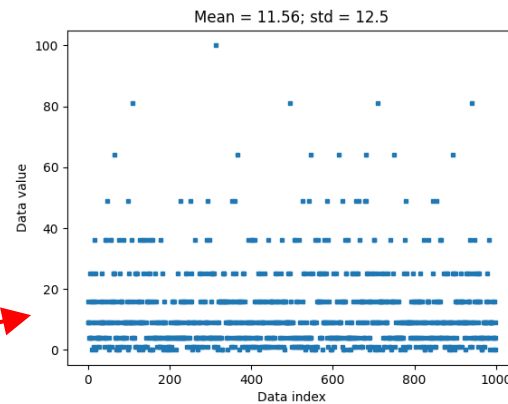
plt.plot(dataz,'s',markersize=3)
plt.xlabel('Data index')
plt.ylabel('Data value')
plt.title(f'Mean = {np.round(datazmean,2)}; std = {np.round(datazstd,2)}')

plt.show()

## show that the relative values are preserved

plt.plot(data,dataz,'s')
plt.xlabel('Original')
plt.ylabel('Z-transformed')
plt.title(f'Correlation r = %g'%np.corrcoef(data,dataz)[0,0])
plt.show()

```

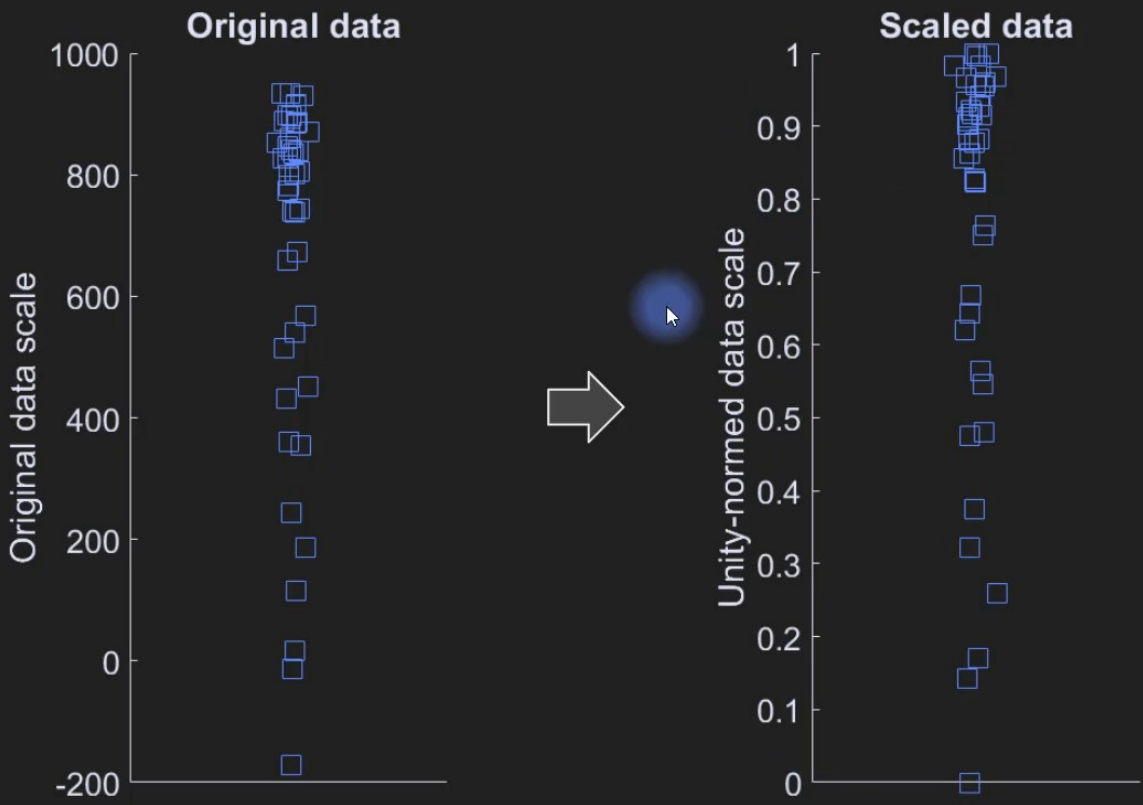


Min-max scaling

In this video, you will learn

- How to scale data to any arbitrary range (typically $[0,1]$).
- **Why min-max scaling is useful.**

Min-max scaling: the main idea



Min-max scaling: the formula

Scale to a range of 0 to 1. $\tilde{x} = \frac{x - \min x}{\max x - \min x}$

Scale to a range of a to b . $x^* = a + \tilde{x}(b - a)$

```
import matplotlib.pyplot as plt
import numpy as np

## create some data
N = 42
data = np.log(np.random.rand(N))*234 + 934
# get min and max
dataMin = min(data)
dataMax = max(data)
# now min-max scale
dataS = (data-dataMin) / (dataMax-dataMin)
# now plot
fig,ax = plt.subplots(1,2,figsize=(8,4))
ax[0].plot(1+np.random.randn(N)/20,data,'ks')
ax[0].set_xlim([0,2])
ax[0].set_xticks([])
ax[0].set_ylabel('Original data scale')
ax[0].set_title('Original data')
```

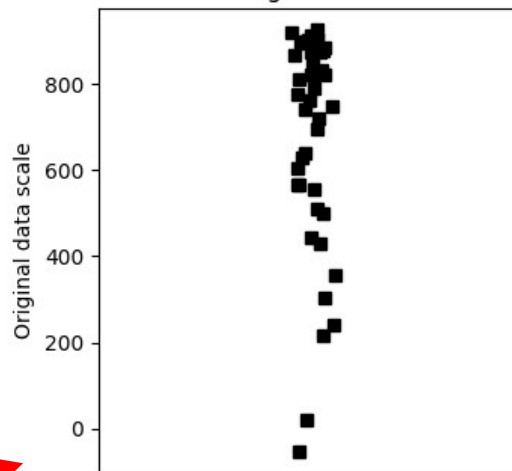
```
ax[1].plot(1+np.random.randn(N)/20,dataS,'ks')
ax[1].set_xlim([0,2])
ax[1].set_xticks([])
ax[1].set_ylabel('Unity-normed data scale')
ax[1].set_title('Scaled data')

plt.show()
```

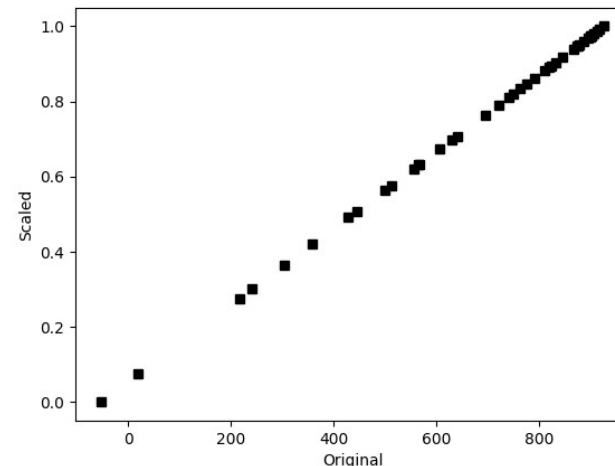
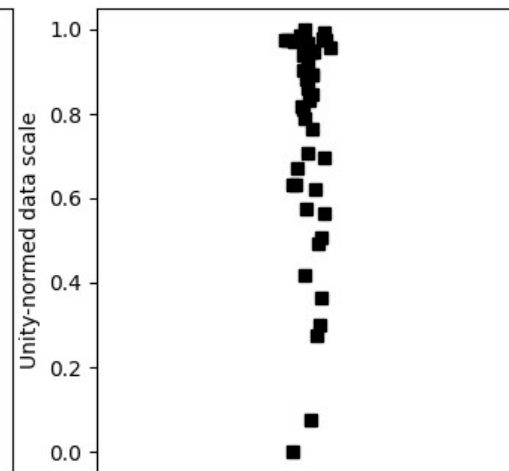
```
## show that scaling doesn't affect the relative values
plt.plot(data,dataS,'ks')
plt.xlabel('Original')
plt.ylabel('Scaled')
plt.show()
```

```
## any arbitrary data range
# step 1 is to [0,1] normalize as above
# step 2:
newMin = 4
newMax = 8.7
dataSS = dataS*(newMax-newMin) + newMin
# test it!
print([min(dataSS), max(dataSS)])
```

Original data



Scaled data



warnings:
[4.0, 8.7]

Min-max scaling: the full formula

$$x^* = a + \left(\frac{x - \min x}{\max x - \min x} \right) (b - a)$$

Your mission!

- 1) Check my formula
- 2) Solve for x
- 3) Implement in code

What are outliers and why are they dangerous?

In this video, you will learn

- Why “outlier” is an ambiguous term.
- What “leverage” means.
- Various strategies for dealing with outliers.
- Why outliers just might be the most important part of your research.

An outlier by another other name

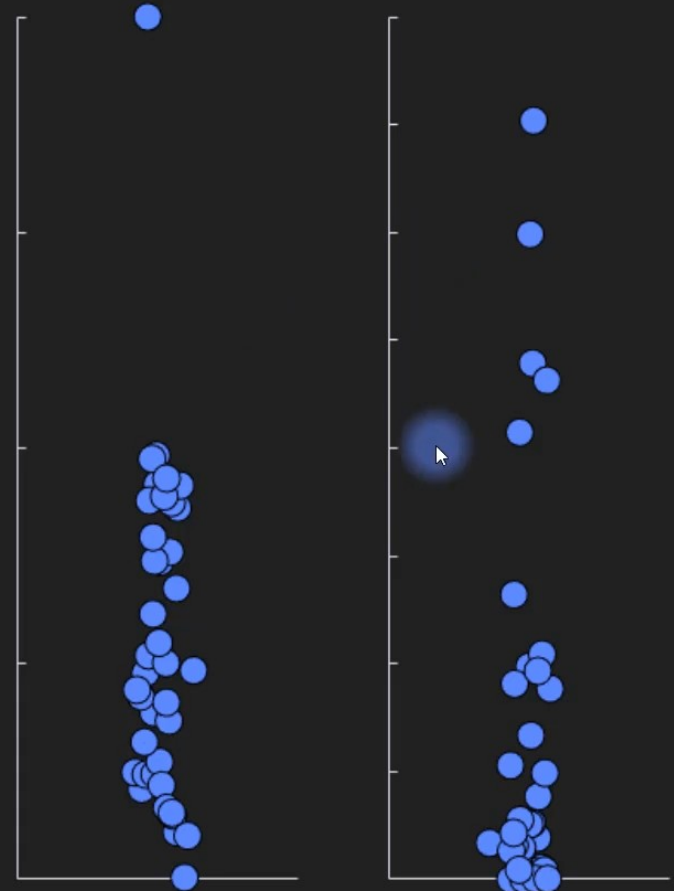
Outlier

Anomaly

Extreme (deviant) data

Non-representative data

Noise



— An outlier by another other name —

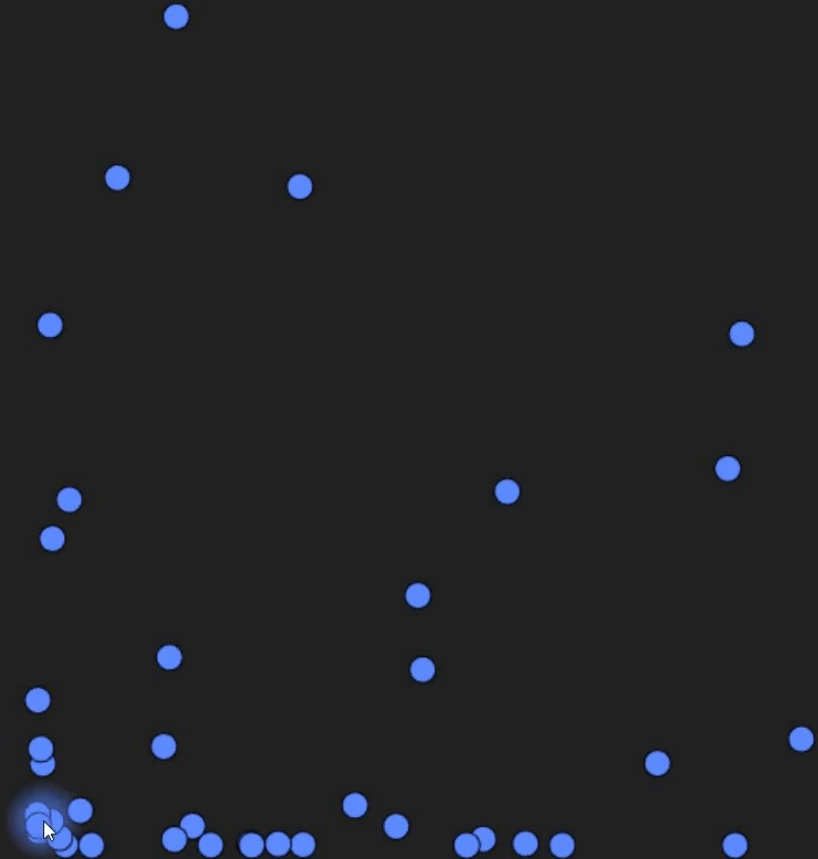
Outlier

Anomaly

Extreme (deviant) data

Non-representative data

Noise



Where do outliers come from?

Noisy data

Noisy or faulty equipment

Human error (e.g., typo)

Non-cooperative research participant

Natural variation

Why are outliers bad?

Many statistical analyses use squared terms (variance, ANOVA, polynomials, GLM, correlation, etc.).

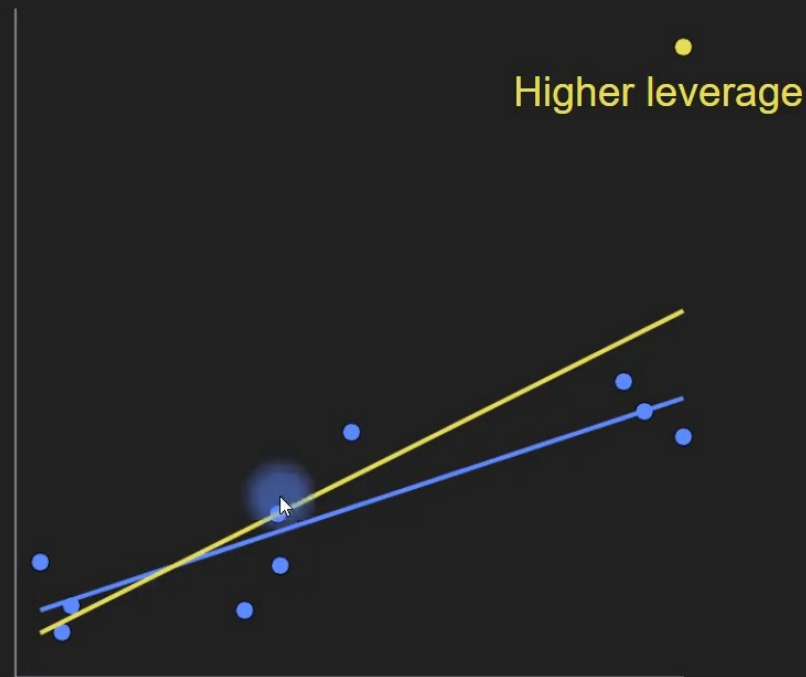
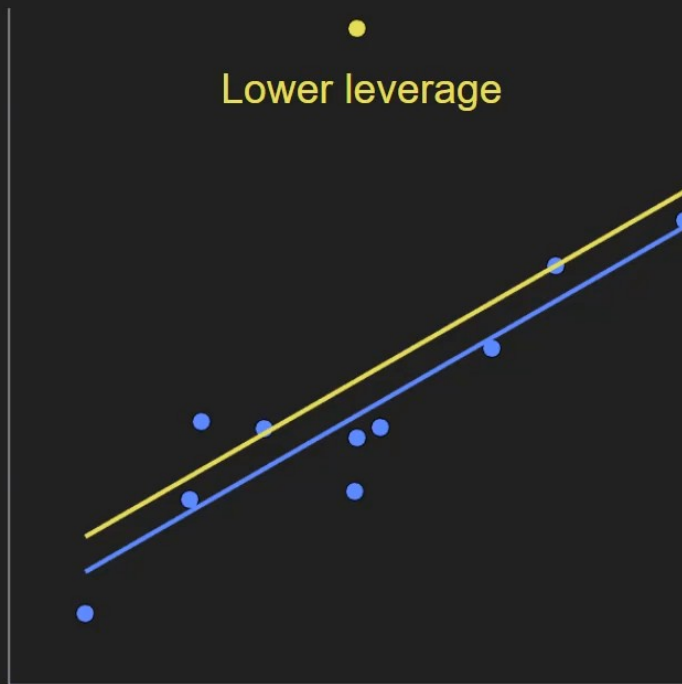
Large outliers become HUGE when squared.

Outliers can have more impact with small N .



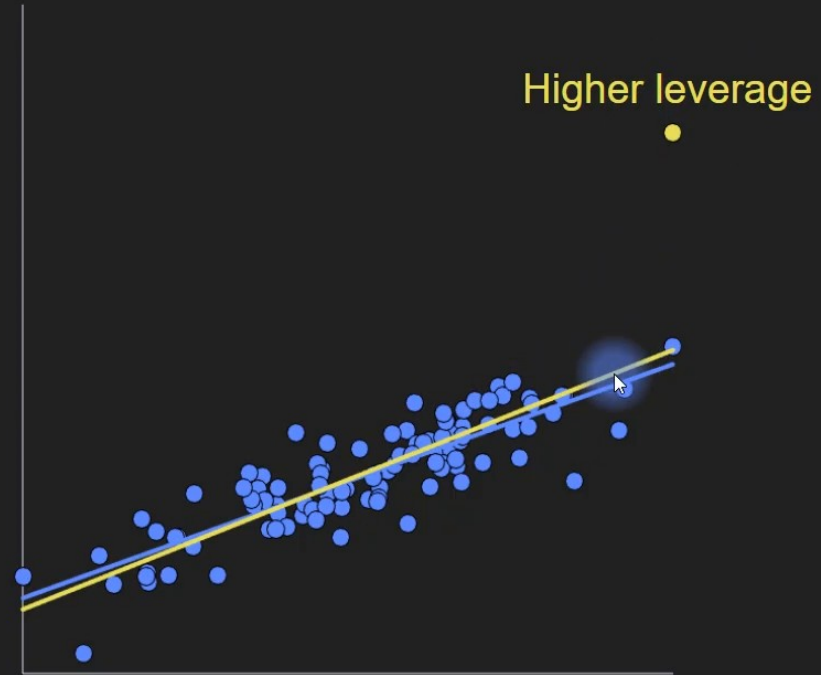
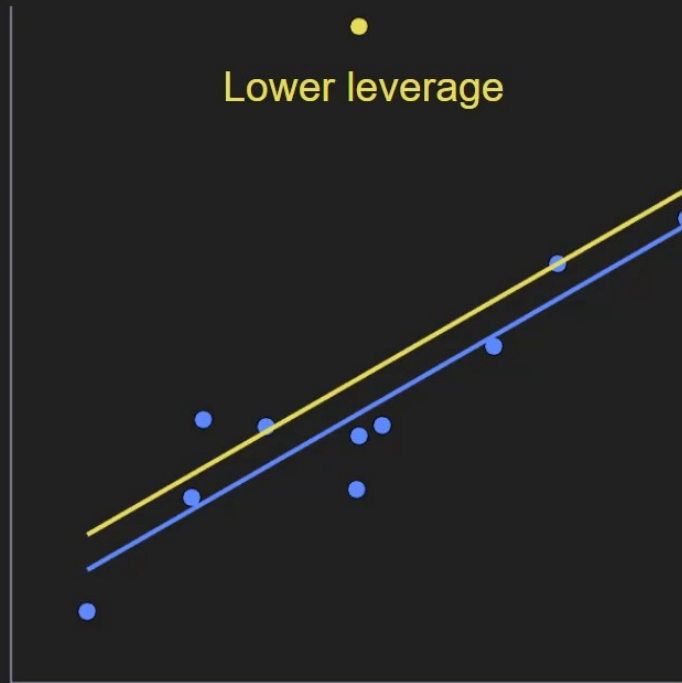
Leverage

Not all outliers are equal. Outliers are worse near the “edges” of the data distribution compared to the “middle.”



Leverage

Not all outliers are equal. Outliers are worse near the “edges” of the data distribution compared to the “middle.”



How to deal with outliers

Strategy 1:

Identify outliers and remove them from the data prior to any analyses.

Assumption: Outliers are noise or otherwise invalid.

Strategy 2:

Leave outliers in and use robust methods that attenuate the negative impact of the outliers on the results.

Assumption: Outliers are unusual but valid data.

How to deal with outliers

Conclusions:

Outliers must be investigated and evaluated.

Never remove outliers without thought.

Never ignore outliers.

Removing outliers: the z-score method

In this video, you will learn

- Quick reminder of the super-important z-score.
- How to use z-scores to identify outliers.
- The iterative z-score outlier method.

The z-score

$$z_i = \frac{x_i - \bar{x}}{\sigma_x}$$

Mean-center and standard-deviation normalized.

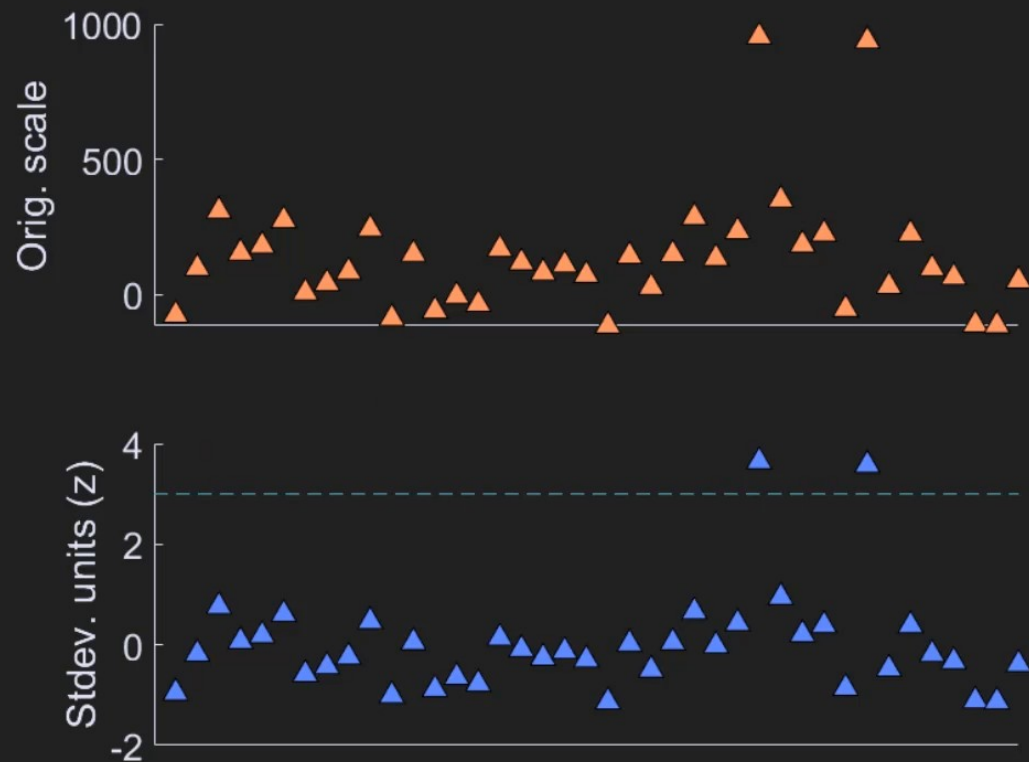
Z-scores are interpreted as standard deviation units away from the center of the distribution.

Z-scores are valid if the distributions are roughly Gaussian.

Z-score for outliers

Algorithm:

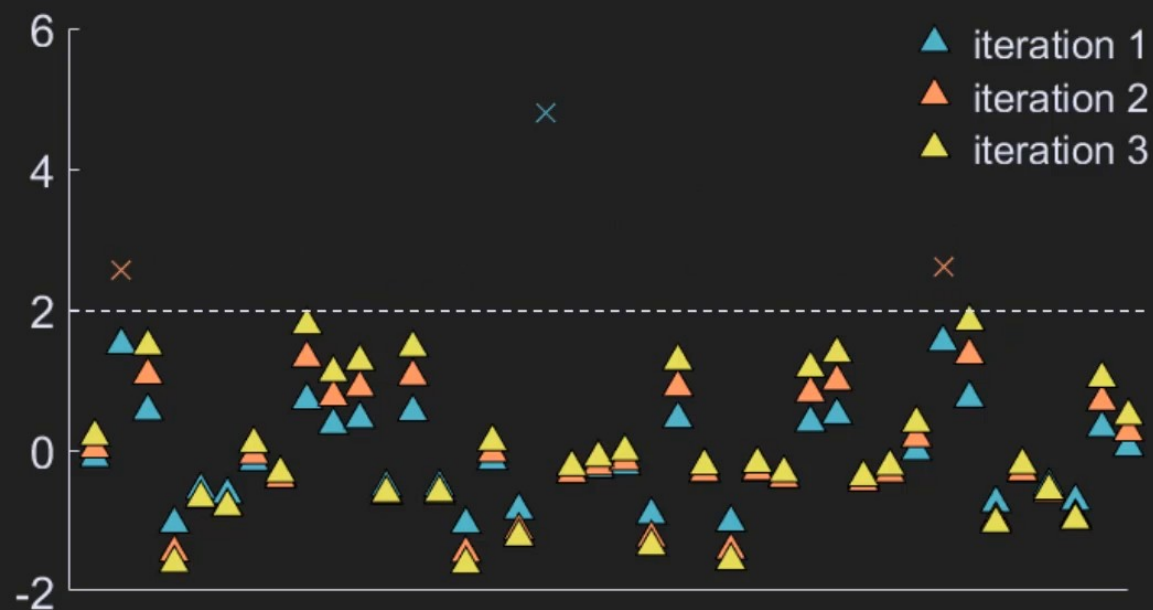
1. Convert data to z-score.
2. A datapoint is an outlier if it exceeds some standard deviation threshold (often 3, but this is arbitrary).



Z-score for outliers

Iterative algorithm:

1. Convert data to z-score.
2. A datapoint is an outlier if it exceeds some standard deviation threshold (often 3, but this is arbitrary).
3. Remove outliers and repeat until no more outliers.



The modified z-score method

In this video, you will learn

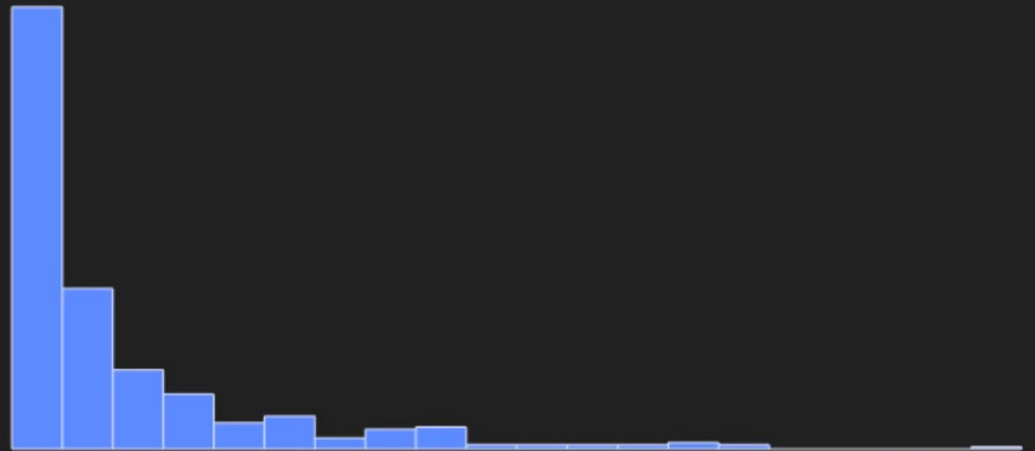
- An alternative to the z-score for non-normal data.

Modified Z-score for outliers

For non-normal distributions:

1. Replace “regular” z-score with the modified z-score.
2. Repeat previous methods.

Useful for long-tailed distributions.



Modified Z-score for outliers

$$M_i = \frac{.6745(x_i - \tilde{x})}{\text{MAD}}$$

$$\text{MAD} = \text{median}(|x_i - \tilde{x}|)$$

$$\tilde{x} = \text{median}(x)$$

```

import numpy as np
import matplotlib.pyplot as plt
from statsmodels import robust
import scipy.stats as stats

## create some data
N = 40
data = np.random.randn(N)
data[data<-1] = data[data<-1]+2
data[data>2] = data[data>2]**2; # try to force a few outliers
data = data*200 + 50 # change the scale for comparison with z

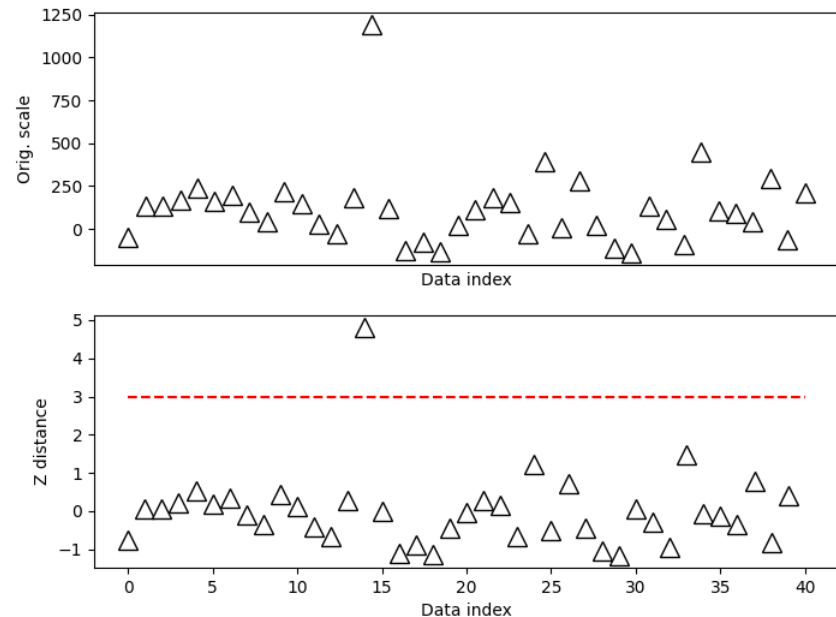
# convert to z
dataZ = (data-np.mean(data)) / np.std(data,ddof=1)
#### specify the z-score threshold
zscorethresh = 3

# plot the data
fig,ax = plt.subplots(2,1,figsize=(8,6))

ax[0].plot(data,'k^',markerfacecolor='w',markersize=12)
ax[0].set_xticks([])
ax[0].set_xlabel('Data index')
ax[0].set_ylabel('Orig. scale')

# then plot the zscores
ax[1].plot(dataZ,'k^',markerfacecolor='w',markersize=12)
ax[1].plot([0,N],[zscorethresh,zscorethresh], 'r--')
ax[1].set_xlabel('Data index')
ax[1].set_ylabel('Z distance')
plt.show()

```



```

## identify outliers
# find 'em!
outliers = np.where(abs(dataZ)>zscorethresh)[0]
# and cross those out
ax[0].plot(outliers,data[outliers],'x',color='r',markersize=20)
ax[1].plot(outliers,dataZ[outliers],'x',color='r',markersize=20)
fig

```

```

## iterative method
# pick a lenient threshold just for illustration
zscorethresh = 2
dataZ = (data-np.mean(data)) / np.std(data,ddof=1)
colorz = 'brkm'
numiters = 0 # iteration counter

```

```

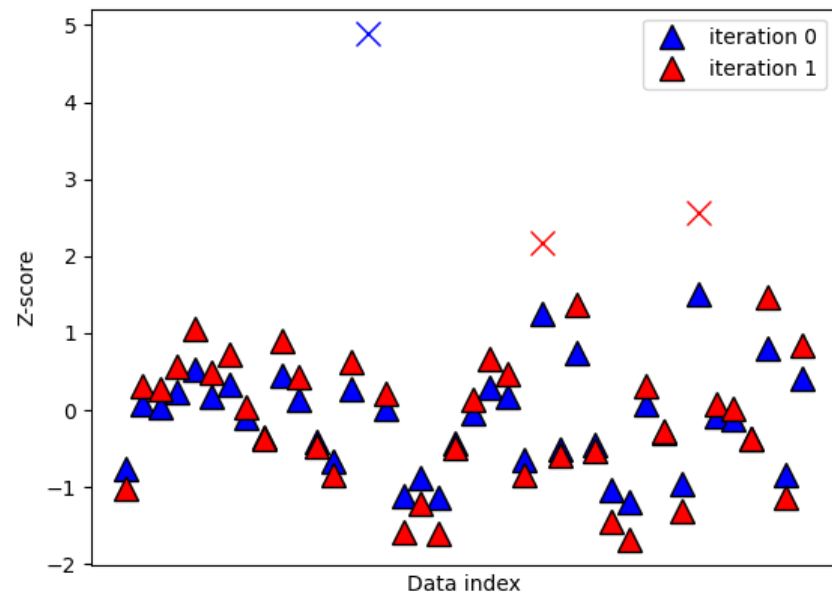
while True:
    # convert to z
    datamean = np.nanmean(dataZ)
    datastd = np.nanstd(dataZ)
    dataZ = (dataZ-datamean) / datastd
    # find data values to remove
    toremove = dataZ>zscorethresh
    # break out of while loop if no points to remove
    if sum(toremove)==0:
        break
    else:
        # otherwise, mark the outliers in the plot
        plt.plot(np.where(toremove)[0],dataZ[toremove], '%sx'%colorz[numiters],markersize=12)
        dataZ[toremove] = np.nan
    # replot
    plt.plot(dataZ,'k^',markersize=12,markerfacecolor=colorz[numiters],label='iteration %g'%numiters)
    numiters = numiters + 1

```

```

plt.xticks([])
plt.ylabel('Z-score')
plt.xlabel('Data index')
plt.legend()
plt.show()

```



```
#### the data points to be removed
removeFromOriginal = np.where(np.isnan(dataZ))[0]
print(removeFromOriginal)
```

[14 24 33]

```
## modified Z for non-normal distributions
```

```
# compute modified z
```

```
dataMed = np.median(data)
```

```
dataMAD = robust.mad(data)
```

```
# note: robust.mad already includes the normalization factor discussed in the video
```

```
dataMz = (data-dataMed) / dataMAD
```

```
# plot the data
```

```
fig,ax = plt.subplots(2,1,figsize=(8,6))
```

```
ax[0].plot(data,'k^',markerfacecolor='w',markersize=12)
```

```
ax[0].set_xticks([])
```

```
ax[0].set_xlabel('Data index')
```

```
ax[0].set_ylabel('Orig. scale')
```

```
# then plot the zscores
```

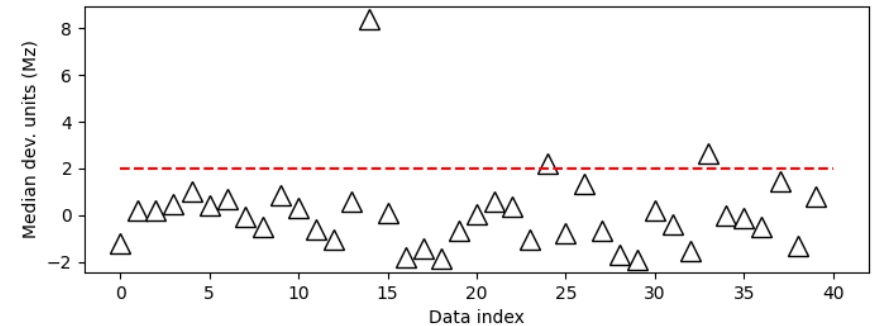
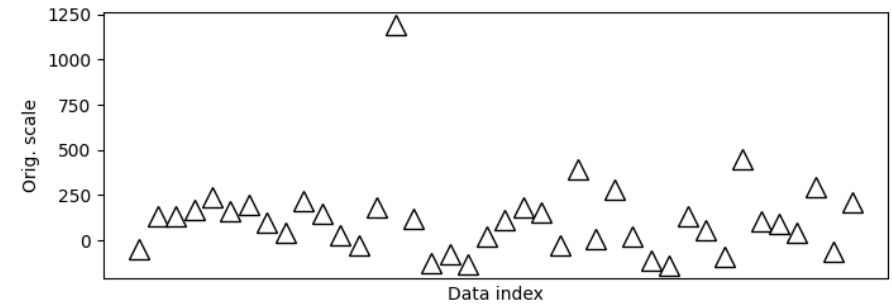
```
ax[1].plot(dataMz,'k^',markerfacecolor='w',markersize=12)
```

```
ax[1].plot([0,N],[zscorethresh,zscorethresh], 'r--')
```

```
ax[1].set_xlabel('Data index')
```

```
ax[1].set_ylabel('Median dev. units (Mz)')
```

```
plt.show()
```

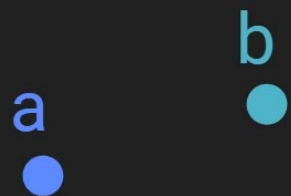


Multivariate outlier detection

In this video, you will learn

- What “Euclidean distance” means.
- How to adapt the z-score method to identify multivariate outliers.

Euclidean distance



$$d_{a,b} = \sqrt{(a_x - b_x)^2 + (a_y - b_y)^2}$$

$$d_{a,b} = \sqrt{(a_x - b_x)^2 + (a_y - b_y)^2 + (a_z - b_z)^2}$$

Multivariate outlier detection

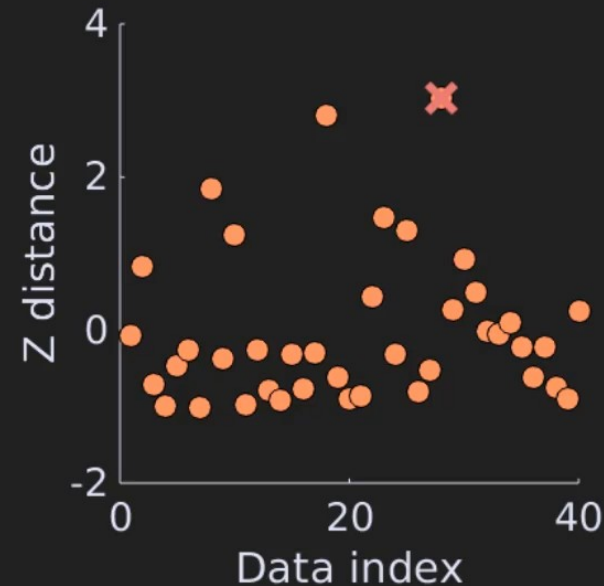
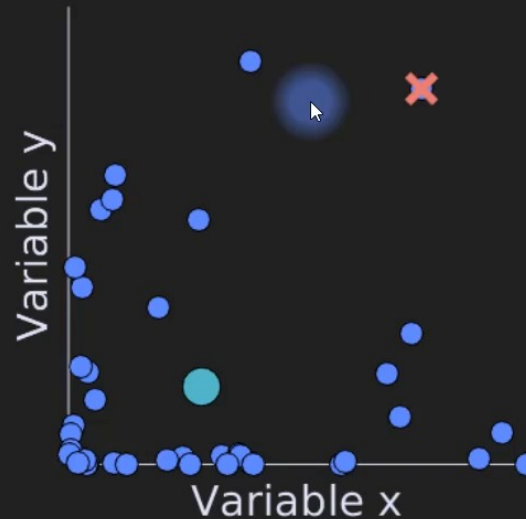


$$d_{a,b} = \sqrt{(a_x - b_x)^2 + (a_y - b_y)^2}$$

Z-score method for multivariate data

Multivariate algorithm:

1. Compute the data mean.
2. Compute the distance from each data point to the mean.
3. Convert distances to Z-score.
4. Remove outliers based on threshold, as shown previously.



```

import numpy as np
import matplotlib.pyplot as plt

## create some data
N = 40
# two-dimensional data
d1 = np.exp(-abs(np.random.randn(N)*3))
d2 = np.exp(-abs(np.random.randn(N)*5))
datamean = [ np.mean(d1), np.mean(d2) ]
# compute distance of each point to the mean
ds = np.zeros(N)
for i in range(N):
    ds[i] = np.sqrt( (d1[i]-datamean[0])**2 + (d2[i]-datamean[1])**2 )
# convert to z (don't need the original data)
ds = (ds-np.mean(ds)) / np.std(ds,ddof=1)
# plot the data
fig,ax = plt.subplots(1,2,figsize=(8,6))

ax[0].plot(d1,d2,'ko',markerfacecolor='k')
ax[0].set_xticks([])
ax[0].set_yticks([])
ax[0].set_xlabel('Variable x')
ax[0].set_ylabel('Variable y')

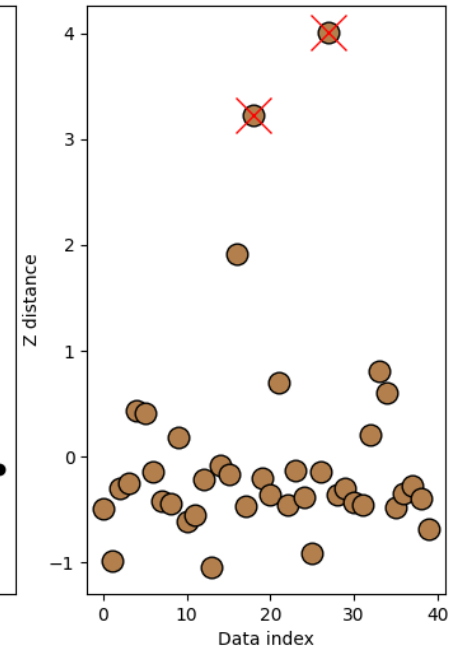
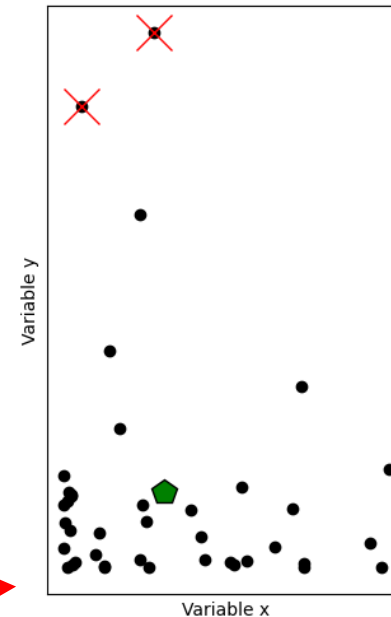
# plot the multivariate mean
ax[0].plot(datamean[0],datamean[1],'kp',markerfacecolor='g',markersize=15)

# then plot those distances
ax[1].plot(ds,'ko',markerfacecolor=[.7, .5, .3],markersize=12)
ax[1].set_xlabel('Data index')
ax[1].set_ylabel('Z distance')

## now for the thresholding
# threshold in standard deviation units
distanceThresh = 2.5
# find the offending points
oidx = np.where(ds>distanceThresh)[0]
print(oidx)
# and cross those out
ax[1].plot(oidx,ds[oidx],'x',color='r',markersize=20)
ax[0].plot(d1[oidx],d2[oidx],'x',color='r',markersize=20)
fig
plt.show()

```

Warning
[18 27]



Removing outliers by data trimming

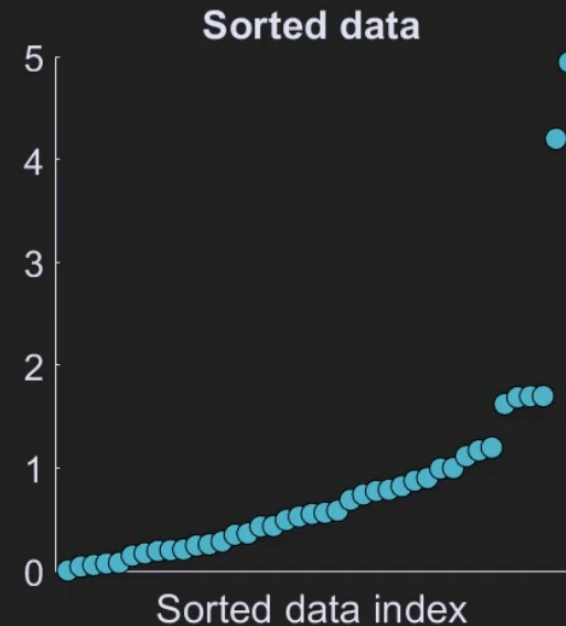
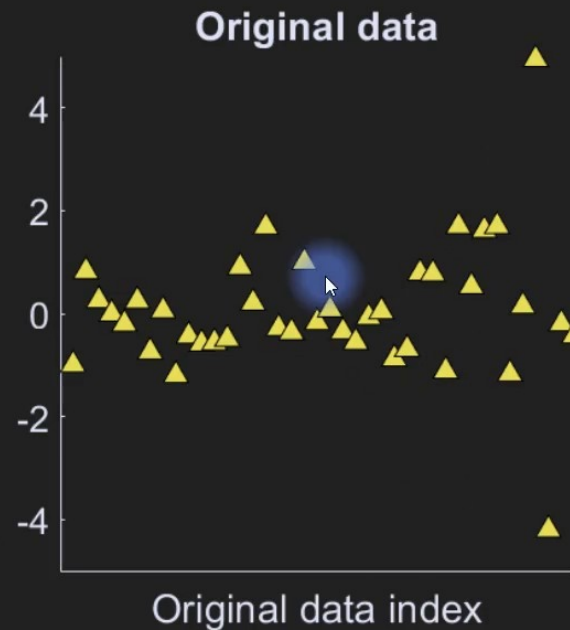
In this video, you will learn

- The simple data trimming maneuver.
- The advantages and limitations of data trimming.

Data trimming: How it works

Algorithm:

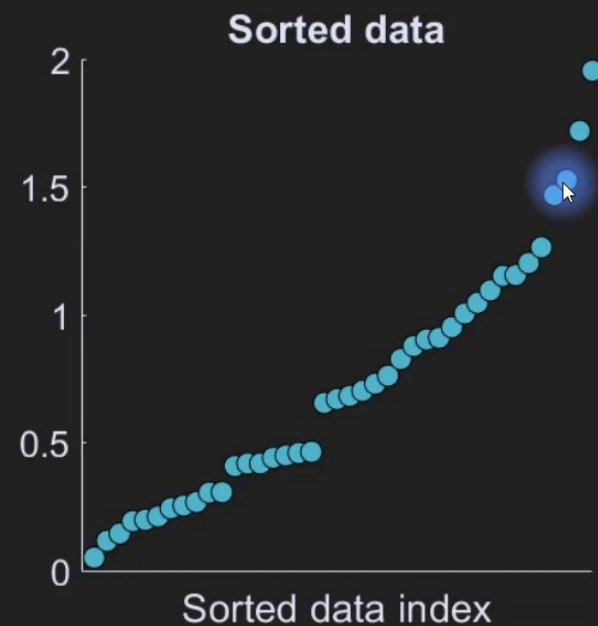
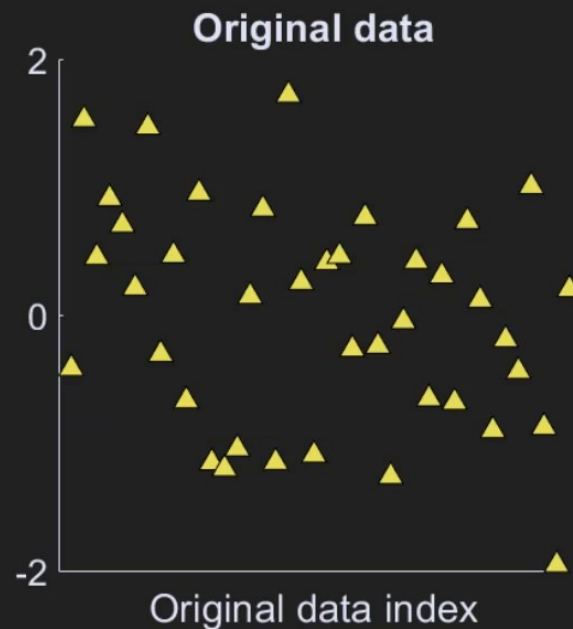
1. Sort the mean-centered data.
2. Remove the most extreme k values, or the most extreme $k\%$.



Data trimming: How it works

Algorithm:

1. Sort the mean-centered data.
2. Remove the most extreme k values, or the most extreme $k\%$.



Pros and cons of data trimming

Advantages

- Simple and easy to implement.
- Can be effective

Disadvantages

- Requires subjective threshold.
- Can remove non-outliers.

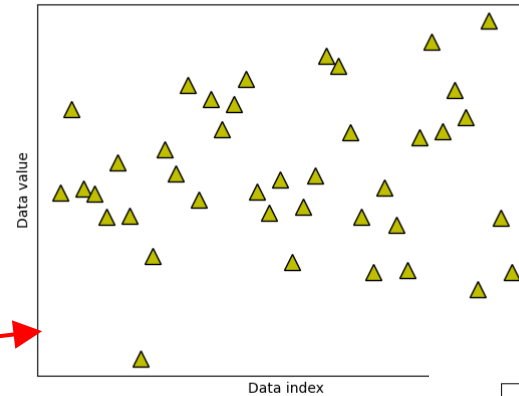
```

import numpy as np
import matplotlib.pyplot as plt

## create some data
N = 40
data = np.random.randn(N)
data[data<-2] = -data[data<-2]**2
data[data>2] = data[data>2]**2

# also need the mean-centered data
dataMC = data - np.mean(data)
# and plot them (not it ;) )
fig,ax = plt.subplots(1,1)
ax.plot(data,'k^',markerfacecolor='y',markersize=12)
ax.set_xticks([])
ax.set_yticks([])
ax.set_xlabel('Data index')
ax.set_ylabel('Data value')
plt.savefig('Figure1.png')

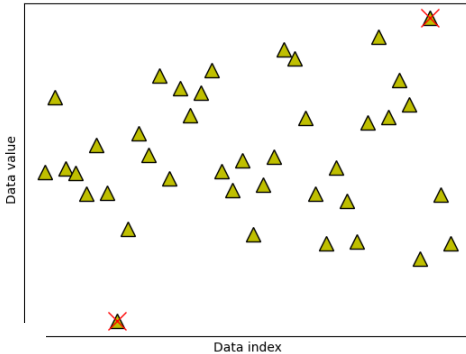
```



```

## option 1: remove k% of the data
# percent of "extreme" data values to remove
trimPct = 5 # in percent
# identify the cut-off (note the abs() )
datacutoff = np.percentile(abs(dataMC),100-trimPct)
# find the exceedance data values
data2cut = np.where( abs(dataMC)>datacutoff )[0]
# and mark those off
ax.plot(data2cut,data[data2cut],'rx',markersize=15)
fig
plt.savefig('Figure2.png')

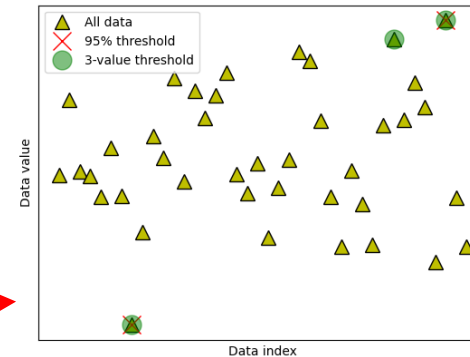
```



```

## option 2: remove k most extreme values
# number of "extreme" data values to remove
k2remove = 3 # in number
# find the exceedance data values
datasortIdx = np.argsort(abs(dataMC),axis=0)[::-1]
data2cut = np.squeeze(datasortIdx[:k2remove])
# and mark those off
ax.plot(data2cut,data[data2cut],'go',markersize=15,alpha=.5)
# finally, add a legend
ax.legend(('All data','g-g-value threshold'%(100-trimPct),'g-value threshold'%k2remove))
fig
plt.savefig('Figure3.png')

```



How to deal with outliers

Strategy 1:

Identify outliers and remove them from the data prior to any analyses.

Assumption: Outliers are noise or otherwise invalid.

Strategy 2:

Leave outliers in and use robust methods that attenuate the negative impact of the outliers on the results.

Assumption: Outliers are unusual but valid data.

Non-parametric solutions to outliers

In this video, you will learn

- A non-exhaustive list of non-parametric analyses that are robust to outliers.
- The difference in philosophy between removing outliers vs. dealing with them in analyses.

Parametric and non-parametric complements

Parametric test

1-sample t-test

2-sample t-test

Pearson correlation

ANOVA

Nonparametric test

Wilcoxon sign-rank test

Mann-Whitney U test

Spearman correlation

Kruskal-Wallis test

Permutation testing

Why are these methods robust to outliers?

They are based on medians or ranks, which are insensitive to outliers.

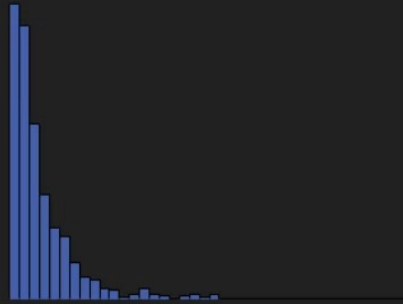
Nonlinear data transformations

In this video, you will learn

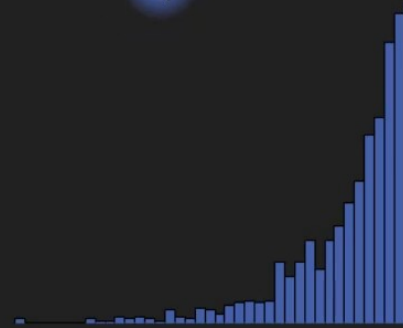
- A few alternatives for making your data more “normal.”
- A warning about interpreting nonlinearly transformed data.

Non-normal data distributions

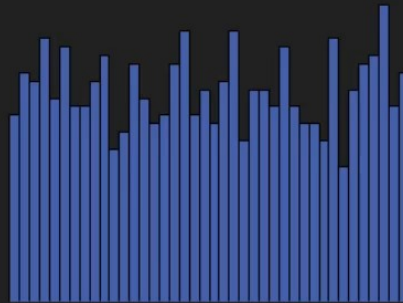
log-normal



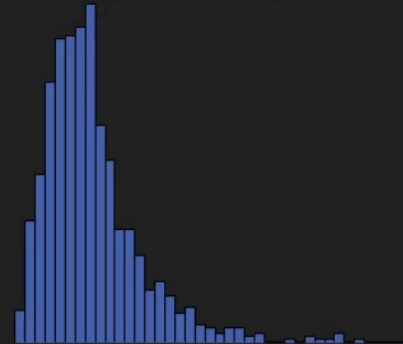
neg. exp.



uniform



generalized extreme



Goal of nonlinear transforms

Many statistical methods are linear or make assumptions about data distributions (e.g., Gaussian).

Non-extreme data values may be labeled as “outliers” due to nonlinear scaling.

Goal: Transform data to make linear methods valid, or to make data distribution approach Gaussian.

Common nonlinear data transformations

Rank-transform

The rank transform

$$s = \{ 4.001, 1, -10, 4, 3948523456293847623984576 \}$$

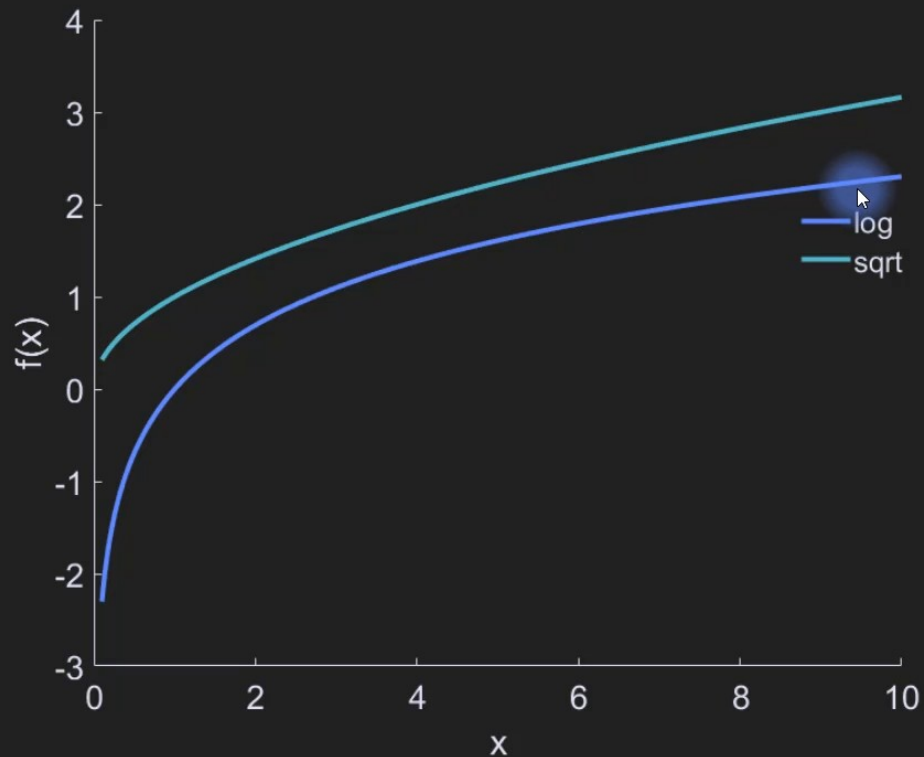
$$s_r = \{ 4, 2, 1, 3, 5 \}$$

Common nonlinear data transformations

Rank-transform

Logarithm

Square root



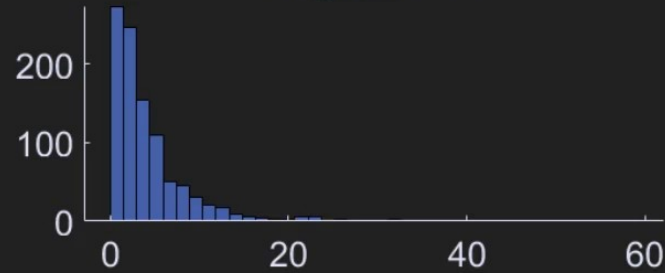
Common nonlinear data transformations

Rank-transform

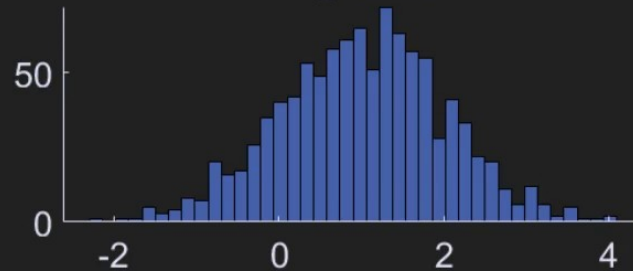
Logarithm

Square root

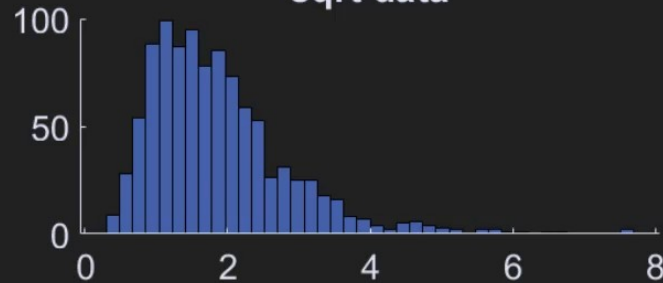
Original data



log data



sqrt data



Common nonlinear data transformations

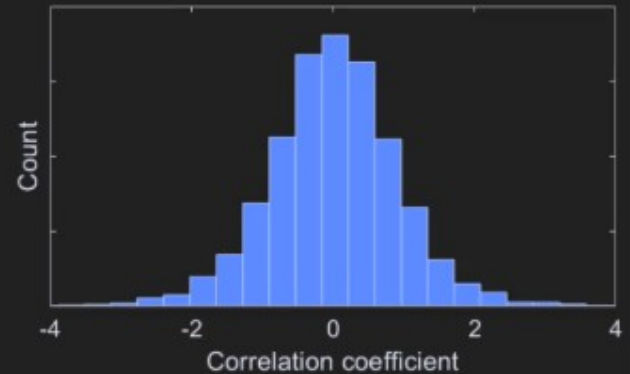
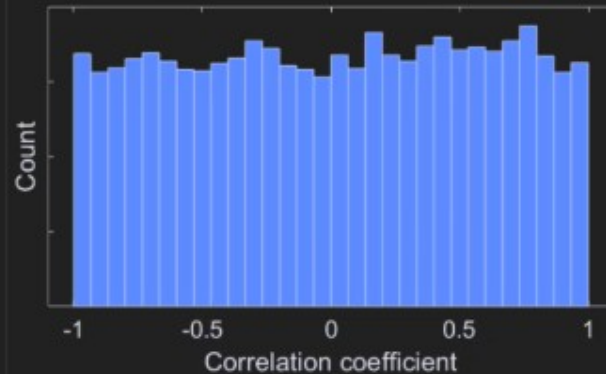
Rank-transform

Logarithm

Square root

Fisher-z

Why do correlations need a transformation?



Mind the interpretation gap

Nonlinear methods are not always appropriate (e.g., negative numbers for log or square root).

Nonlinear transformations alter the spacing between data points as a function of data value.

Most statistical models are linear, so results must be interpreted in terms of the transformed data, not the original data.